Circles-lbp Audit

Côme du Crest

2025-22-01

Table of contents

- Table of contents
- Cicles-lbp Audit
  - Scope
  - Context
  - Status
- Issues
  - [High][Fixed] No slippage parameter on exitLBP
  - [High][Fixed] No slippage parameter on CoW swap
  - [Med][Ack] Varying pool weights of low frequency pools is dangerous
  - [Info][Ack] Backing asset amount can be inflated
  - Optimisations and miscellaneous

Cicles-lbp Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes all contracts within `aboutcircles/circles-lbp` as of commit `0xe35ec6a`.

The code has been audited a second time as of commit `0xe35a779`.

Context

The goal of the contracts is to create a balancer pool from user-provided circles tokens (`CRC`) and another accepted asset (`wBTC`, `wETH`, `GNO`, `sDAI`) for which the LP tokens are locked for a year.

Status

The report has been sent to the core developer.

Code has been updated and all issues have been either fixed or acknowledged. The last audited commit is `0xe35a779`.

Issues

[High][Fixed] No slippage parameter on exitLBP()

Summary

There is no slippage parameter on the helper function `exitLBP()` that exits the pool and accepts any amounts of out tokens.

Vulnerability Detail

The function `CirclesBackingFactory.exitLBP()` uses 0 values as minimum amounts to receive when exiting the given pool:

```
    function exitLBP(address lbp, uint256 bptAmount) external {
        // transfer bpt tokens from msg.sender
        IERC20(lbp).transferFrom(msg.sender, address(this), bptAmount);

        uint256[] memory minAmountsOut = new uint256[](2);
        minAmountsOut[0] = uint256(0);
        minAmountsOut[1] = uint256(0);

        bytes32 poolId = ILBP(lbp).getPoolId();

        (IERC20[] memory poolTokens,,) = IVault(VAULT).getPoolTokens(poolId);
        if (poolTokens.length != minAmountsOut.length) revert
            OnlyTwoTokenLBPSupported();

        // exit pool
        IVault(VAULT).exitPool(
            poolId,
            address(this), // sender
            payable(msg.sender), // recipient
            IVault.ExitPoolRequest(
                poolTokens, minAmountsOut, abi.encode(ILBP.ExitKind.
                    EXACT_BPT_IN_FOR_TOKENS_OUT, bptAmount), false
            )  // @audit no slippage parameter
        );
    }
```

Impact

A call to `exitLBP()` can be front-run to put the pool in an unfavourable state before the exit is applied.

The caller will receive a lesser proportion of the valuable asset of the pool resulting in a loss.

Code Snippets

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L245-L267

Recommendation

Remove this function or accept minimal amounts as parameter and compute acceptable value off-chain.

Response

Fixed in commit `0x74ab0ba` by having expected min amounts as input parameters of `exitLBP()`.

[High][Fixed] No slippage parameter on CoW swap

Summary

When creating the pool, USDC are swapped to `backingAsset` using CoW swap. The generated order uses `buyAmount = 1` meaning the order is valid with as little as one token received. This can be abused by any counterparty to profit from the trade.

Vulnerability Detail

The function `CirclesBackingFactory.startBacking()` generates the orderUid with `buyAmount = 1` and sends the orderUid to `CirclesBacking`:

```
    function startBacking(address backer, address backingAsset, address
        stableCRCAddress, uint256 stableCRCAmount)
        internal
    {
        ...
        bytes memory orderUid = generateOrderUID(instance, backingAsset, 1);
            // @audit no slippage param
        // Initiate cowswap order
        CirclesBacking(instance).initiateCowswapOrder(orderUid);
        ...
    }
```

The value `buyAmount` represents the amount of tokens the recipient of the order is accepting:

```
    function generateOrderUID(address instance, address backingAsset, uint256
        buyAmount)
        public
        view
        returns (bytes memory orderUid)
    {
        // create order
        (, bytes32 appData) = getAppData(instance);
        // Generate order UID using the "getUid" contract
        (bytes32 orderDigest,) = GET_UID_CONTRACT.getUid(
            USDC, // sellToken
            backingAsset, // buyToken
            instance, // receiver
            TRADE_AMOUNT, // sellAmount
            buyAmount, // buyAmount
            VALID_TO, // order expiry
            appData, // appData hash
            0, // FeeAmount
            true, // IsSell
            false // PartiallyFillable
        );
        // Construct the order UID
        orderUid = abi.encodePacked(orderDigest, instance, uint32(VALID_TO));
```

```
23         }
```

Impact

Anyone seeing the order being created by monitoring the event `OrderCreated(orderUid)` on contracts created by `CirclesBackingFactory` can react by creating an inverse order selling 1 `backingAsset` token for the total `TRADE_AMOUNT` defined in the factory. The two matching orders should be matched by CoW solvers resulting in a loss to circles-lbp users.

Code Snippets

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L180

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L335

Recommendation

Use a valid `buyAmount` value either provided by the user in the `onERC1155Received()` or by computing an expected received amount with on-chain oracles (TWAP or Chainlink).

Response

The `CirclesBackingFactory` uses `ValueFactory` which uses off-chain oracles to compute an expected amount of backing asset to receive. In case the oracle for backing asset is stale, a minimum amount of 1 is still accepted.

[Med][Ack] Varying pool weights of low frequency pools is dangerous

Summary

Varying the balancer pool weights over time may have negative impact on the price if trade frequency is low.

Moreover, the USD value of inflationary CRC tokens decreases over time due to the nature of circles tokens. However the balancer pool weights of `backingAsset/CRC` evolve from `99/1` to `50/50` over the course of one year. This means that with regards to the pool if no trades are done CRC tokens are seen as more valuable over time.

Vulnerability Detail

The `CRC/backingAsset` pool is created by `CirclesBackingFactory` with a `1/99` ratio:

```
    function createLBP(address personalCRC, uint256 personalCRCAmount, address
        backingAsset, uint256 backingAssetAmount)
        external
        returns (address lbp, bytes32 poolId, IVault.JoinPoolRequest memory
            request, address vault)
    {
        address backer = onlyCirclesBacking();
        // prepare inputs
        IERC20[] memory tokens = new IERC20[](2);
        bool tokenZero = personalCRC < backingAsset;
        tokens[0] = tokenZero ? IERC20(personalCRC) : IERC20(backingAsset);
        tokens[1] = tokenZero ? IERC20(backingAsset) : IERC20(personalCRC);

        uint256[] memory weights = new uint256[](2);
        weights[0] = tokenZero ? WEIGHT_1 : WEIGHT_99;
        weights[1] = tokenZero ? WEIGHT_99 : WEIGHT_1;

        // create LBP
        lbp = LBP_FACTORY.create(
            _name(personalCRC),
            _symbol(personalCRC),
            tokens,
            weights,
            SWAP_FEE,
            msg.sender, // lbp owner
            true // enable swap on start
        );

        ...
    }
```

The pool weights are updated by `CirclesBacking` to a `50/50` weight ratio over one year:

```
    function createLBP() external {
```

```
2           ...
3           // update weight gradually
4           uint256 timestampInYear = block.timestamp + UPDATE_WEIGHT_DURATION;
5           ILBP(lbp).updateWeightsGradually(block.timestamp, timestampInYear,
                _endWeights());
6
7           // set bpt unlock
8           balancerPoolTokensUnlockTimestamp = timestampInYear;
9       }
10
11      function _endWeights() internal pure returns (uint256[] memory endWeights)
            {
12          endWeights = new uint256[](2);
13          endWeights[0] = WEIGHT_50;
14          endWeights[1] = WEIGHT_50;
15      }
```

Impact

Assume that the pool is initiated with 48 USDC and 48 CRC and no trade take place over the course of one year. At the pool creation time, 1 USDC can be trade for:

$$CRC_{out} = B_{CRC} * (1 - \frac{B_{USDC}}{B_{USDC} + USDC_{in}}^{\frac{0.99}{0.01}}) = 41.77CRC \tag{1}$$

After one year, when the weights are at a 50/50 ratio if the balances of the pool did not change 1 USDC can be traded for:

$$CRC_{out} = B_{CRC} * (1 - \frac{B_{USDC}}{B_{USDC} + USDC_{in}}^{\frac{0.50}{0.50}}) = 0.97CRC \tag{2}$$

If the pool is meant to be rarely used and provide a price oracle for the CRC, varying the weights of the pool is an incorrect approach. The varying weights of the pool are meant to vary exposure to certain assets for liquidity providers. I am unsure this is the behaviour intended by the developer.

Code Snippets

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L200-L206

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBacking.sol#L141

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBacking.sol#L171-L175

Recommendation

Use a stable weight pool. Using a weight of `50/50` or at most `10/90` is recommended.

Response

The issue has been acknowledged with comment: “Varying pool weights is a core feature of Liquidity Bootstrapping Pools and a project requirement. Many LBPs could be considered as a distributed CRC global reserve, we have arbitrage bot in place, which is making related trades between them in order to have average price - it has impact on trade frequency.”

[Info][Ack] Backing asset amount can be inflated

Summary

When creating an LBP, the amount of backing asset to join the pool with is determined by the balance of the `CirclesBacking` contract for which the address can be pre-computed. This amount can be inflated and not match the expected USD value of `TRADE_AMOUNT`.

Vulnerability Detail

The function `CirclesBacking.createLBP()` uses the current balance of the backing asset as `backingAmount` and call the factory's `createLBP` with that amount to compute the `join` request. It later uses that request to join the balancer pool:

```
    function createLBP() external {
        if (lbp != address(0)) revert AlreadyCreated();
        // Check if the order has been filled on the CowSwap settlement
            contract
        uint256 filledAmount = COWSWAP_SETTLEMENT.filledAmount(storedOrderUid);
        address backingAsset;
        uint256 backingAmount;

        if (filledAmount != 0) {
            // use picked backing asset in case cowswap order is executed
            backingAsset = BACKING_ASSET;
            backingAmount = IERC20(backingAsset).balanceOf(address(this));  //
                @audit can be inflated
            if (backingAmount == 0) revert InsufficientBackingAssetBalance();
        } ...

        // Create LBP
        bytes32 poolId;
        IVault.JoinPoolRequest memory request;
        address vault;
        (lbp, poolId, request, vault) = FACTORY.createLBP(STABLE_CRC,
            STABLE_CRC_AMOUNT, backingAsset, backingAmount);

        // approve vault
        IERC20(STABLE_CRC).approve(vault, STABLE_CRC_AMOUNT);
        IERC20(backingAsset).approve(vault, backingAmount);

        // provide liquidity into lbp
        IVault(vault).joinPool(
            poolId,
            address(this), // sender
            address(this), // recipient
            request
        );

        ...
```

```
34      }
```

The function `CirclesBackingFactory.createLBP()` uses the provided `backingAssetAmount` to compute the join request:

```
1       function createLBP(address personalCRC, uint256 personalCRCAmount, address
            backingAsset, uint256 backingAssetAmount)
2           external
3           returns (address lbp, bytes32 poolId, IVault.JoinPoolRequest memory
                request, address vault)
4       {
5           ...
6           uint256[] memory amountsIn = new uint256[](2);
7           amountsIn[0] = tokenZero ? personalCRCAmount : backingAssetAmount;
8           amountsIn[1] = tokenZero ? backingAssetAmount : personalCRCAmount;
9
10          bytes memory userData = abi.encode(ILBP.JoinKind.INIT, amountsIn);
11
12          request = IVault.JoinPoolRequest(tokens, amountsIn, userData, false);
13          ...
14      }
```

Impact

I do not understand the reason behind this project and cannot evaluate whether this behaviour poses a real threat.

Code Snippets

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBacking.sol#L111

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L223-L229

Recommendation

Acknowledge the issue. Otherwise, `CirclesBacking` needs to be aware of the amount of `backingAsset` it received which could be done by using an `isBuy` type of trade (i.e. fixed amount out) as the initial CoW swap order.

Response

This issue has been acknowledged.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

Typos in CirclesBackingFactory errors

"Dissallowed" should be spelled "Disallowed" and "Bootstraping" should be spelled "Bootstrapping".

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L27-L28

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L37C24-L37C36

Response: this has been fixed.

Unnecessary `nonReentrant` modifier

Since I was asked about it, I believe the `nonReentrant` modifier of `onERC1155Received()` is not necessary as only controlled functions are called that cannot re-enter this function (ERC20, circles Hub and Lift, CoWSwap settlement). I do not recommend removing it.

https://github.com/aboutcircles/circles-lbp/blob/e35ec6ad6b5a09ae24062d27f540effb08973260/src/CirclesBackingFactory.sol#L432

Response: the modifier has been kept.